

BOOTCAMP MLOps

BOOTCAMP MLOps:

What we will learn to day:
Experiment Tracking
& Model Management

Danh Pham

Goals

- Answer the question: Why reproducibility is crucial?
- Full components of a models tracking system.
- MLFlow:
  - What is it?
  - Why we need it?
  - How to use it?

Contents

The "Why" - The Reproducibility Crisis

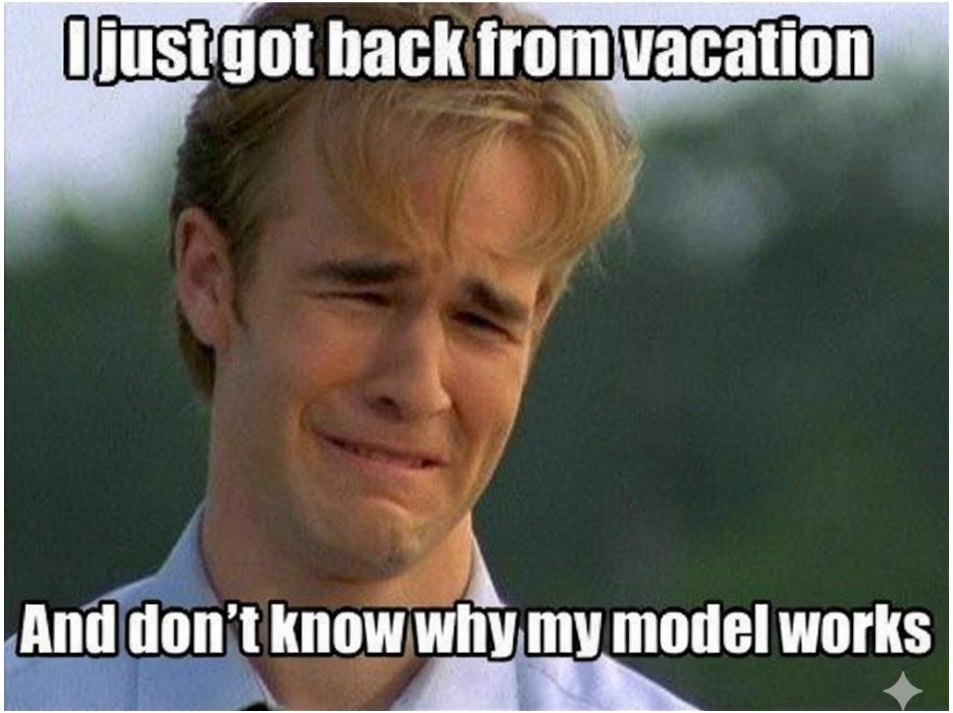
What Constitutes an "Experiment"?

Introducing MLflow & Best Practices

Hands-on Lab

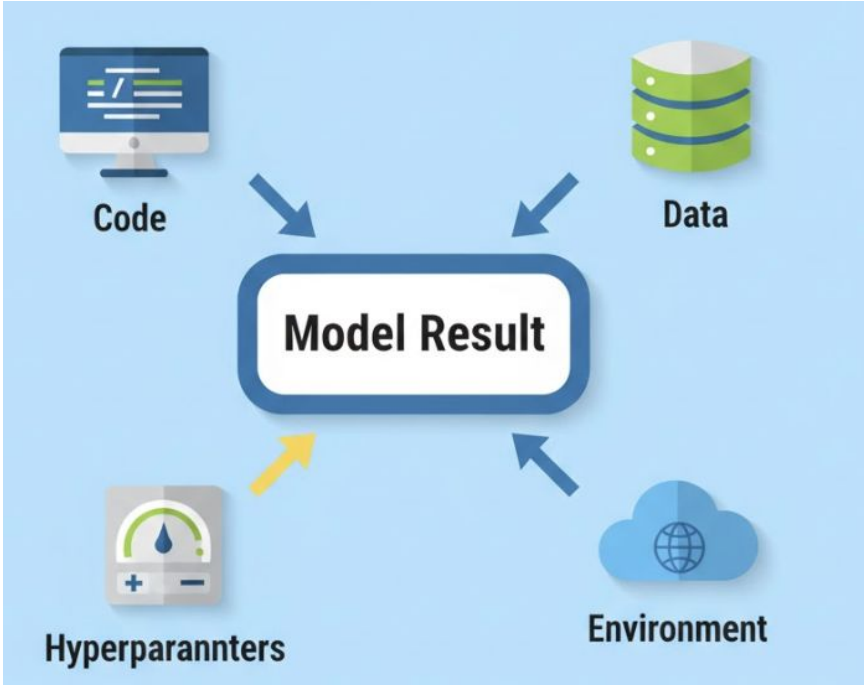
But it used to work…

- Have you ever struggled to recreate a result?
- Weeks or months later, can you reproduce your best model exactly?
- What if a colleague needs to?



The Moving Parts of an ML Project

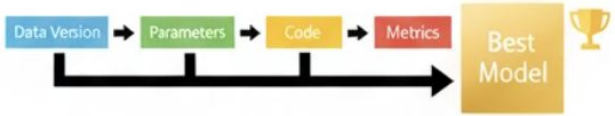
- **Code:** Small changes in preprocessing or training logic.
- **Data:** A different subset, a new version, or a different random split.
- **Hyperparameters:** The secret sauce! (e.g., learning rate, tree depth).
- **Environment:** Library versions (e.g., scikit-learn 1.0 vs. 1.1).



From Chaos to Order

- The goal is to capture everything needed to recreate a result.
- Think of it like a scientist's lab notebook, but for Machine Learning.
- **MLOps Solution:** Use tools to automatically log every component of an experiment.

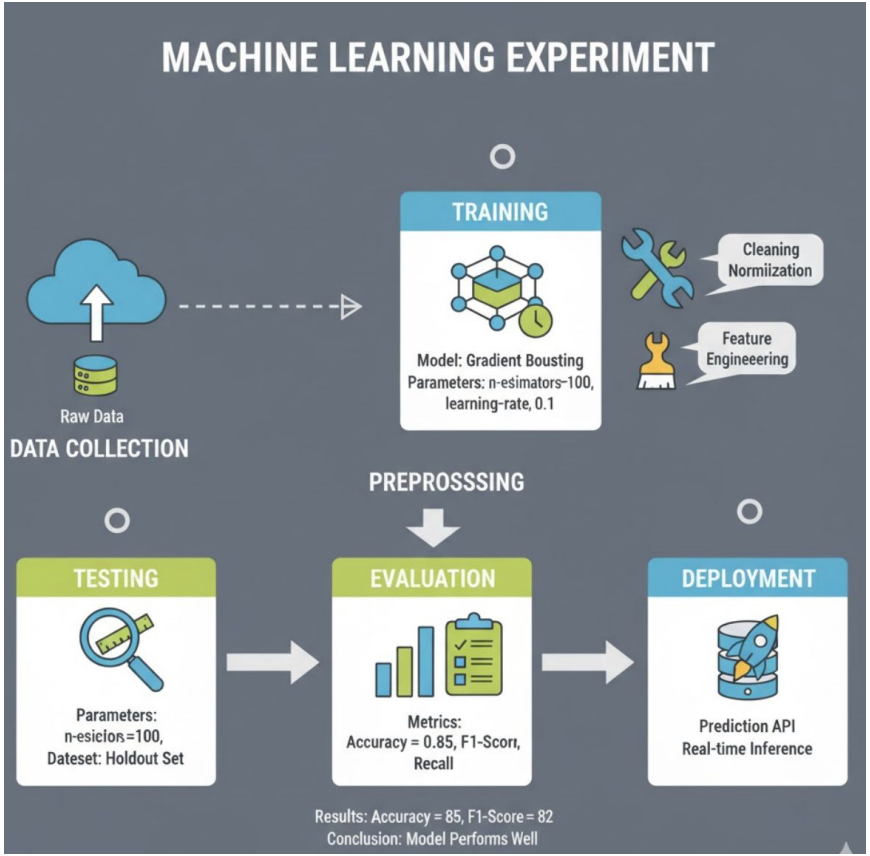




What is an Experiment?

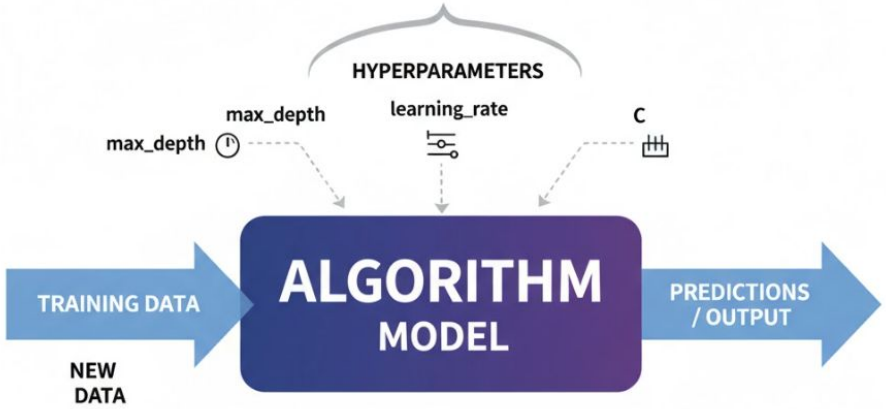
- An "***experiment***" is a single execution of your training script (train.py).

- Our goal is to track the **Inputs** and **Outputs** of every run.



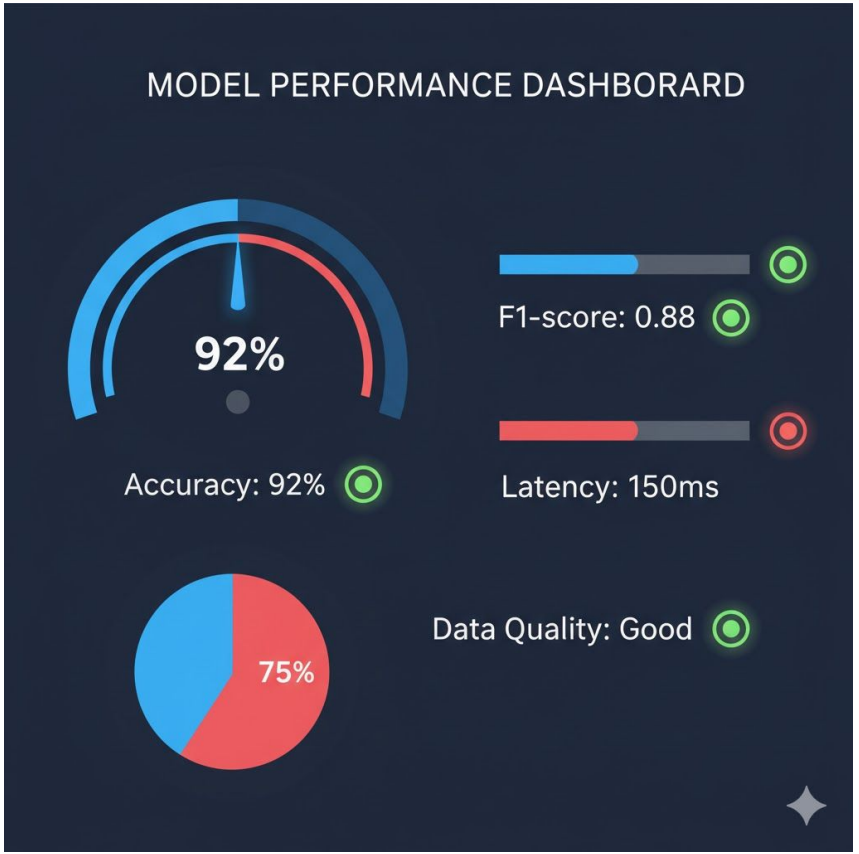
Experiment Inputs: Parameters

- The settings you provide to your algorithm.
- Also known as **Hyperparameters**.
- Examples:
  - max_depth in a Decision Tree
  - learning_rate in a neural network
  - C in a Support Vector Machine
  - random_state for reproducibility



Experiment Outputs: Metrics

- The quantitative results of your run.
- A score that tells you how well the model performed.
- Examples:
  - accuracy
  - precision, recall, f1-score
  - mean_squared_error (MSE)



Experiment Outputs: Artifacts

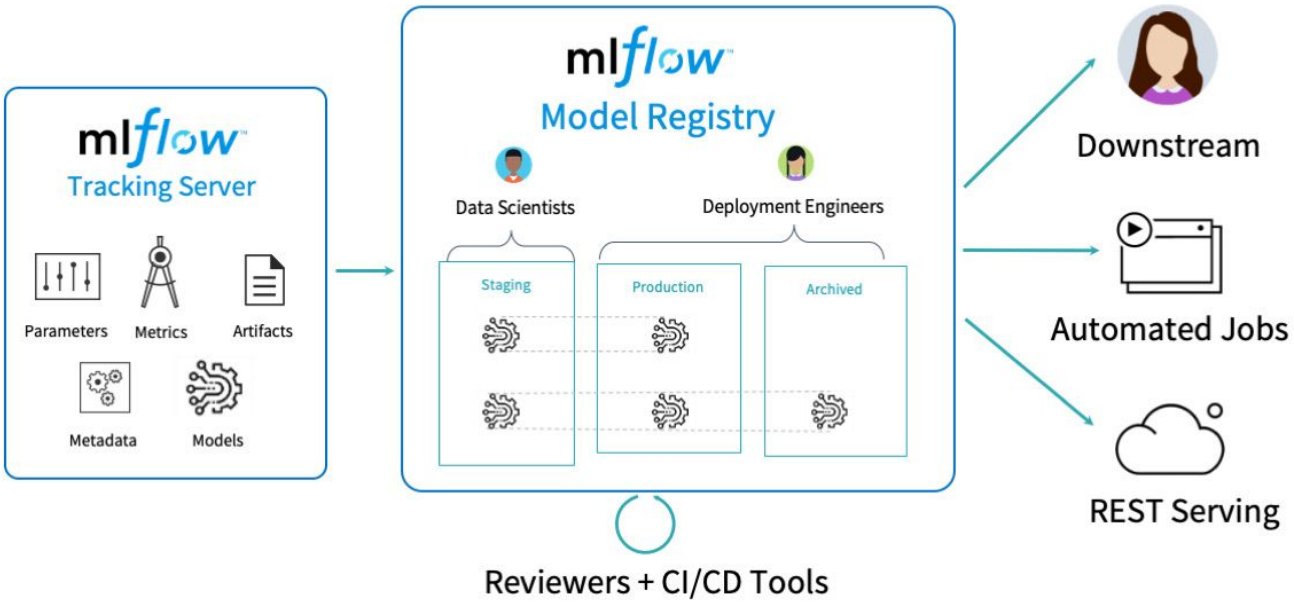
- The files produced by your run.
- Any output that isn't a simple number.
- Examples:
  - The model file itself (model.joblib)
  - Plots (e.g., confusion matrix, feature importance)
  - Data files (e.g., a processed version of the input data)



The Anatomy of a Single Run



Introducing MLflow



https://www.databricks.com/blog/2020/04/15/databricks-extends-mlflow-model-registry-with-enterprise-features.html

MLflow Component 1: Tracking

The MLflow Tracking is an API and UI for logging parameters, code versions, metrics, and output files when running your machine learning code and for later visualizing the results. MLflow Tracking provides Python , REST , R, and Java APIs.



https://mlflow.org/docs/2.22.1/tracking/

MLflow Component 1: Tracking - Concepts

Runs

Each *run* represents one execution of your ML code (e.g., train.py).
It logs metadata (metrics, parameters, timestamps) and artifacts (model files, plots, etc.).

Experiments

An *experiment* is a collection of related runs for a specific task.
Use the CLI, API, or UI to create, search, and organize experiments.

Tracing

Tracing captures detailed execution data from GenAI apps.
It helps you review, debug, and analyze traces via MLflow's Tracking API and integrations like LangChain.

https://mlflow.org/docs/2.22.1/tracking/

MLflow Component 2: Models

An **MLflow Model** is a standard format for packaging machine learning models that can be used in a variety of downstream tools—for example, real-time serving through a REST API or batch inference on Apache Spark. The format defines a convention that lets you save a model in different “flavors” that can be understood by different downstream tools.

```
# Directory written by
mlflow.sklearn.save_model(model,
"my model")
my model/
├── MLmodel
├── model.pkl
├── conda.yaml
├── python env.yaml
└── requirements.txt
```

MLmodel content

```
time_created: 2018-05-25T17:28:53.35

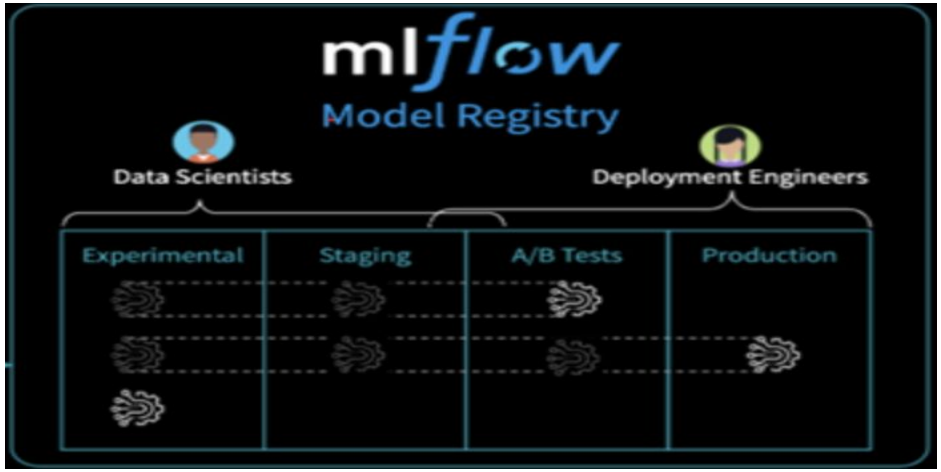
flavors:
  sklearn:
    sklearn version: 0.19.1
    pickled model: model.pkl
  python function:
     loader_module: mlflow.sklearn
```

COLE

https://mlflow.org/docs/2.3.0/models.html

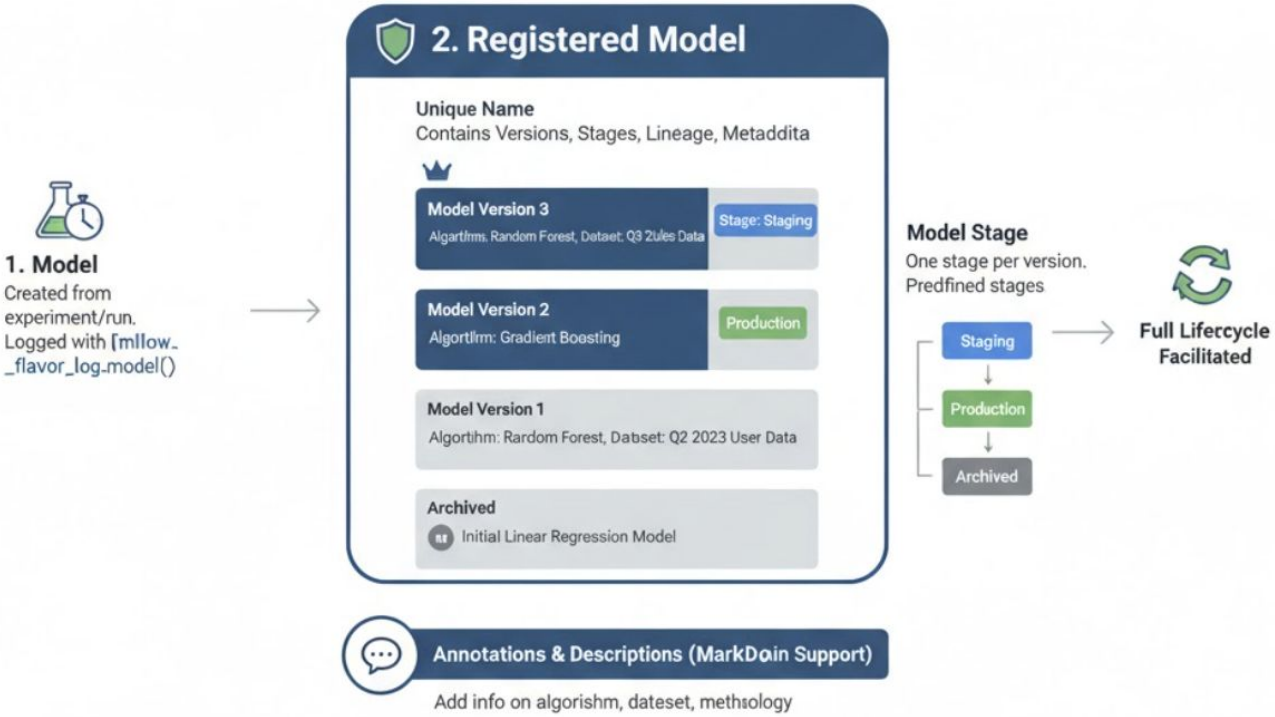
MLflow Component 3: Model Registry

The **MLflow Model Registry** component is a centralized model store, set of APIs, and UI, to collaboratively manage the full lifecycle of an MLflow Model. It provides model lineage (which MLflow experiment and run produced the model), model versioning, stage transitions (for example from staging to production), and annotations.



https://mlflow.org/docs/2.3.0/model-registry.html#

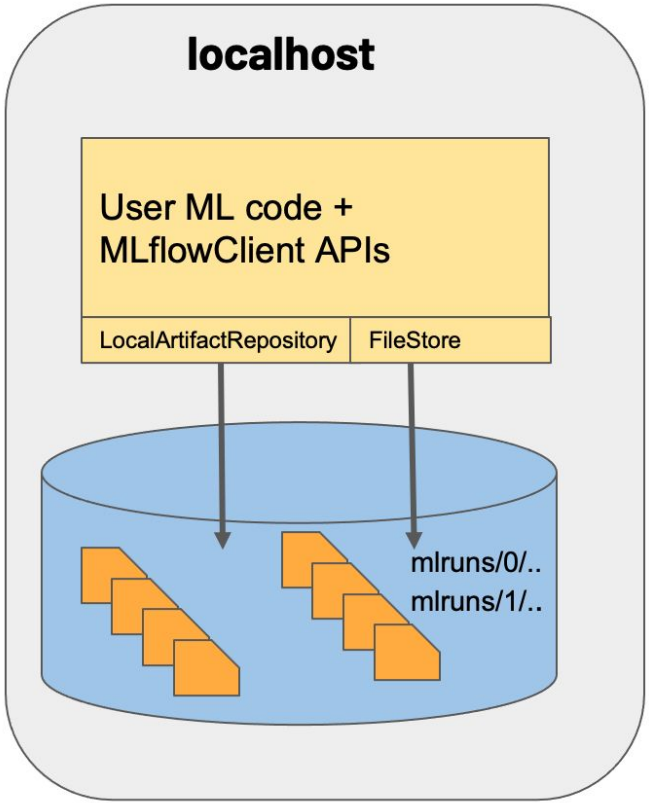
MLflow Component 3: Model Registry - Concepts



https://mlflow.org/docs/2.3.0/model-registry.html#

How MLflow Works (Locally)

Both the backend and artifact store share a directory on the local filesystem—**./mlruns**—as shown in the diagram. The MLflow client directly interfaces with an instance of a **FileStore** and **LocalArtifactRepository**.



https://mlflow.org/docs/2.3.0/tracking.html#id33

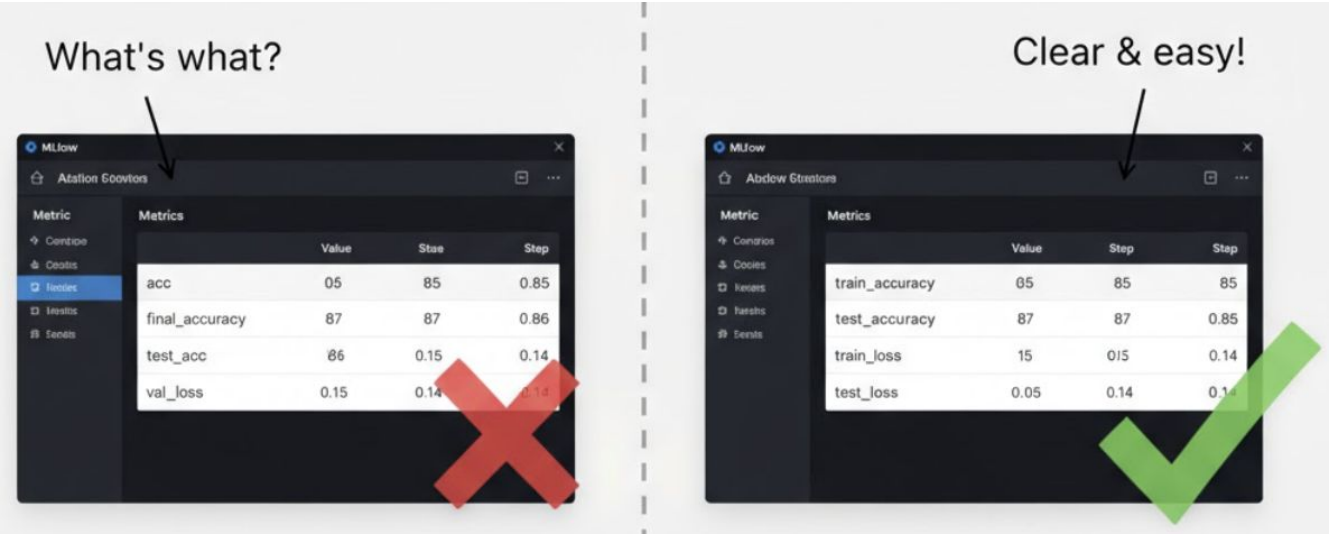
The MLflow Workflow



https://mlflow.org/docs/2.3.0/tracking.html#id33

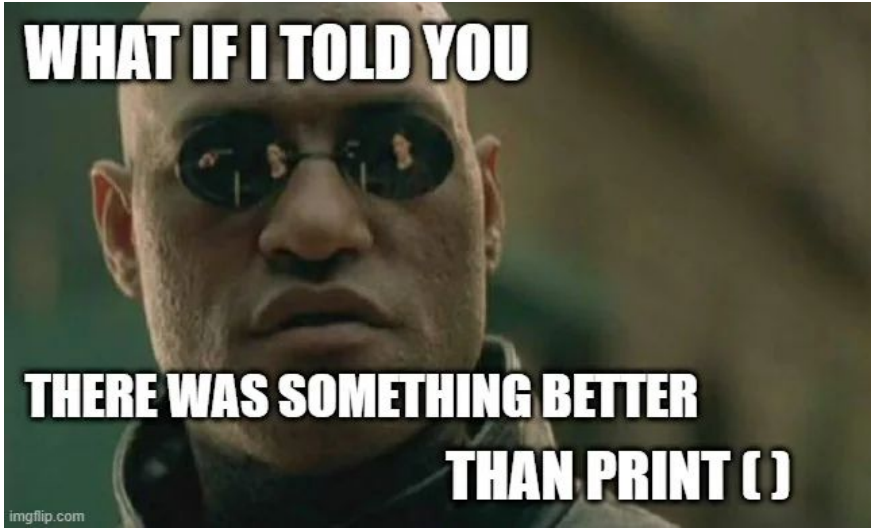
Best Practices: Consistency is Key

- Use consistent, self-explanatory names.
- **Bad:** acc, final_acc, test_score
- **Good:** training_accuracy, validation_accuracy, test_accuracy
- This makes comparison in the UI much more effective.



Best Practices: Log Everything Important

- **Rule of thumb:** If you had to change it or if it affects the result, log it.
- **Examples:**
  - Feature set used (features: "v1" vs. "v2")
  - Random seed (random_state: 42)
  - Training duration (training_time_seconds: 120.5)
  - A short note or tag (mlflow.set_tag("note", "Using normalized features"))



COLE

The Entry Point: mlflow.start_run()

```
import mlflow

with mlflow.start_run():
    # ... your training code here ...

    mlflow.log_param("my_param", 5)
    mlflow.log_metric("my_metric", 0.95)
```



COLE

https://mlflow.org/docs/2.3.0/tutorials-and-examples/tutorial.html#id5

The Core Logging Functions

- **Log a parameter:**
  - `mlflow.log_param("param_name", value)`
- **Log a metric:**
  - `mlflow.log_metric("metric_name", value)`
- **Log a model (artifact):**
  - `mlflow.sklearn.log_model(model, "model_folder_name")`
- **Log any other file (artifact):**
  - `mlflow.log_artifact("path/to/your/file.png")`

COLE

https://mlflow.org/docs/2.3.0/tutorials-and-examples/tutorial.html#id5

Your mission

- **Goal:** Integrate MLflow into our train.py script.
- **Steps:**
  1. Install MLflow.
  2. Add logging code.
  3. Run multiple experiments.
  4. Compare results in the UI.
  5. Register our best model.

COLE

Q&A



Bài tập thực hành

MLflow Tracking defaults to logging run data to local files, which can be frustrating due to scattered files and poor access. Alternatively, Python users can use SQLite (e.g., mlruns.db) for local file system storage with a built-in client, avoiding extra dependencies and server setup.

Follow this guide:
https://mlflow.org/docs/3.1.3/ml/tracking/tutorials/local-database/

Key Takeaways

1. **Reproducibility is Foundational** 🔄: Track experiments to capture parameters, code, and data to reproduce model results.
2. **MLflow Logs Everything** 🛠️: Instrument code with MLflow to log input parameters, output metrics, and the model as an artifact for each training run.
3. **Use the UI to Compare Runs** 📊: Launch MLflow UI to compare experiment metrics and parameters to find the best one.
4. **Registry Manages Model Versions** 🏷️: Utilize MLflow Model Registry to version, manage, and track the lifecycle of high-performing models.

COLE

Thank you for listening!

